



Wallet Application Security Audit Report



Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

3.3 Vulnerability Summary

4 Audit Result

5 Statement

1 Executive Summary

On 2026.08.03, the SlowMist security team received the Rabby team's security audit application for Rabby Mobile Wallet iOS, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "black-box and grey-box" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for wallet application includes two steps:

The codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The wallet application is manually analyzed to look for any potential issues.

The following is a list of security audit items considered during an audit:

NO.	Audit Items	Result
1	App runtime environment detection	Passed
2	Code decompilation detection	Passed
3	App permissions detection	Passed
4	File storage security audit	Passed
5	Communication encryption security audit	Passed
6	Interface security audit	Passed
7	Business security audit	Passed
8	WebKit security audit	Passed
9	App cache security audit	Passed
10	WebView DOM security audit	Passed
11	SQLite storage security audit	Passed

NO.	Audit Items	Result
12	Deeplinks security audit	Passed
13	Client-Based Authentication Security audit	Passed
14	Signature security audit	Passed
15	Deposit/Transfer security audit	Passed
16	Transaction broadcast security audit	Passed
17	Secret key generation security audit	Passed
18	Secret key storage security audit	Passed
19	Secret key usage security audit	Passed
20	Secret key backup security audit	Passed
21	Secret key destruction security audit	Passed
22	Screenshot/screen recording detection	Passed
23	Paste copy detection	Passed
24	Keyboard keystroke cache detection	Passed
25	Insecure entropy source security audit	Passed
26	Background obfuscation detection	Passed
27	Suspend evoke security audit	Passed
28	AML anti-money laundering security policy detection	Passed
29	Others	Passed
30	User interaction security	Passed
31	Cryptography security audit	Passed

3 Project Overview

3.1 Project Introduction

Audit Version

Mobile Source Code:

<https://github.com/RabbyHub/rabby-mobile/commit/37c094f9fd6327c6dfd96498b4b7313d686965b8>

App Name: rabbymobile.ipa

Sha256: f7c3c1886c081f4aa30c79cd32390927fa0388ca51d71a8cb0c6b26c6812b1c4

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	App Runtime Environment Issue	App runtime environment detection	Suggestion	Acknowledged
N2	Communication Encryption Security Issue	Communication encryption security audit	Suggestion	Acknowledged
N3	Business Security Issue	Business security audit	Suggestion	Acknowledged
N4	Signature Security Issue	Signature security audit	Suggestion	Acknowledged
N5	Screenshot/Screen Recording Issue	Screenshot/screen recording detection	Suggestion	Acknowledged
N6	Keyboard Keystroke Cache Issue	Keyboard keystroke cache detection	Suggestion	Acknowledged
N7	User Interaction Security Issue	User interaction security	Suggestion	Acknowledged

3.3 Vulnerability Summary

[N1] [Suggestion] App Runtime Environment Issue

Category: App runtime environment detection

Content

The Rabby Mobile App lacks detection mechanisms for device jailbreak and Frida Hook.

Solution

It is recommended to add iOS jailbreak detection and Hook detection.

Status

Acknowledged

[N2] [Suggestion] Communication Encryption Security Issue

Category: Communication encryption security audit

Content

1. Communication encryption is implemented via the HTTPS protocol for data transmission.
2. For communication encryption, certificate verification is performed on the client side, and mTLS is not adopted.

Solution

It is recommended to adopt mTLS or certificate whitelisting for communication encryption.

1. For mTLS authentication, it is generally required to verify that the Certificate Authority (CA) is a trusted entity before validating domain-name and public-key information within certificates.
2. mTLS is generally more secure than Server-authenticated TLS. Server-authenticated TLS is vulnerable to bypass, enabling analysis of the application's network requests. Though mTLS is harder to bypass, feasible bypass vectors still exist.

3. Whether Server-authenticated TLS or mTLS is deployed, implementing certificate pinning helps mitigate man-in-the-middle (MITM) attacks. However, during testing, Server-authenticated TLS is comparatively easier to bypass for application traffic analysis.
4. Alternatively, configure a certificate whitelist on the client side to prevent man-in-the-middle attacks.

Status

Acknowledged

[N3] [Suggestion] Business Security Issue

Category: Business security audit

Content

In gasAccount.ts, setGasAccountSig writes sig/accountId in plaintext into the unencrypted mmkv.default (whose protection strength is inconsistent with the encrypted keyringMMKV). With the plaintext sig, seven types of Gas account APIs can be called (query, withdrawal, history, checkGasAccountTx, etc.). However, funds can only be transferred to the current wallet address and cannot be transferred to other addresses. The impact is limited to Gas balance consumption and information disclosure.

Solution

It is recommended to store the sig in encrypted form. Alternatively, do not persist the sig; instead, have the user sign or the wallet perform silent signing each time a gas account is used.

Status

Acknowledged

[N4] [Suggestion] Signature Security Issue

Category: Signature security audit

Content

EIP-712 chainId=0 bypasses the chain consistency check.

When the iOS wallet receives an EIP-712 Typed Data signature request (eth_signTypedData_v3 / eth_signTypedData_v4) initiated by a DApp, it validates that domain.chainId in the message is "consistent with the currently connected chain". This validation logic has a flaw: when domain.chainId = 0, it is bypassed, causing the wallet to be unable to reject signature requests with a chain mismatch. In addition, the validation is also silently passed through when "the current chain fails to parse".

Problem code location (provider.ts, lines 517-527)

```
const signTypedDataVlvalidation = (req) => {
  const {
    data: { params: [from, data] },
    session, account,
  } = req;
  let jsonData;
  try {
    jsonData = JSON.parse(data);
  } catch (e) {
    throw ethErrors.rpc.invalidParams('data is not a validate JSON string');
  }
  if (!dappService.isInternalDapp(session.origin)) {
    const currentChain =
      getProviderRequestChain(req)?.enum ||
      dappService.getDapp(session.origin)?.chainId;

    if (jsonData.domain.chainId) { // ← Problem 1: truthiness check
      const chainItem = findChainByEnum(currentChain);
      if (
        !currentChain ||
        (chainItem && Number(jsonData.domain.chainId) !== chainItem.id) // ←
        Problem 2
      ) {
        throw ethErrors.rpc.invalidParams(
          'chainId should be same as current chainId',
        );
      }
    }
  }
  ...
};
```

Problem 1: if (jsonData.domain.chainId) truthiness check

In JavaScript, 0 is a falsy value. When a malicious DApp submits {"domain":{"chainId":0,...}}:

The outer guard if (jsonData.domain.chainId) evaluates to false.

The entire "chainId consistency validation" block is skipped.

The "chainId should be same as current chainId" error is never thrown.

The request proceeds directly to the subsequent signing flow.

Problem 2: (chainItem && ...) silently passes through when the chain fails to parse

Even if domain.chainId is truthy (non-zero), when findChainByEnum(currentChain) returns undefined (the current chain cannot be parsed):

In chainItem && Number(jsonData.domain.chainId) !== chainItem.id, chainItem is undefined.

The whole expression is undefined (falsy), so the outer if is not satisfied.

Any chainId (including the target chain's real chainId) is passed through.

Solution

It is recommended to check whether the field exists instead of checking its truthiness. The confirmation page should also use the same existence check, and directly reject the request when it does not match. Add regression tests for chainId=0, chainId="0", missing chainId, and a normal chain ID.

Status

Acknowledged

[N5] [Suggestion] Screenshot/Screen Recording Issue

Category: Screenshot/screen recording detection

Content

Screenshot/screen recording detection is missing.



Solution

It is recommended to add screenshot/screen recording detection.

Status

Acknowledged

[N6] [Suggestion] Keyboard Keystroke Cache Issue

Category: Keyboard keystroke cache detection

Content

Password input uses the system keyboard, and the App does not provide its own secure keyboard.

Solution

It is recommended to integrate a custom keyboard in the App to avoid keystroke information being cached by third-party keyboards when entering mnemonic passwords.

Status

Acknowledged

[N7] [Suggestion] User Interaction Security Issue

Category: User interaction security

Content

Functionality	Support	Notes
WYSIWYS	✓	There is friendly parsing of the data.
AML	✓	AML strategy is supported.
Anti-phishing	✓	Phishing detect warning is supported.
Pre-execution	✓	Pre-execution result display is supported.
Contact whitelisting	✓	The contact whitelisting is supported.
Password complexity requirements	✗	The password has no complexity requirements.

Tip: ✓ Full support, ● Partial support, ✗ No support

Solution

It is recommended to enhance user interaction security.

Status

Acknowledged

4 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002608120003	SlowMist Security Team	2026.08.03 - 2026.08.12	Passed

Summary conclusion: The SlowMist security team leveraged manual review alongside SlowMist-developed analytical tools to conduct an audit on the target project. Seven improvement-oriented suggestions were uncovered throughout the auditing procedure, and every single one of these discovered issues has been acknowledged.

5 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>